Week_One_Notes

Dalton Mason

2/22/2021

Reproducible Research: Concepts and Ideas (Part 1)

In order to validate the results of research, replication is the gold standard. For a study to be considered valid, it should be replicated and the same outcome should be observed. In modern times, pure replication has become more difficult due to the amount of money and time that may need to be invested. In these cases, where replication may not be possible, reproducibility is important.

If the original study makes its data and computational methods available, they can be scrutinized. This kind of reproducibility is a validation of the data analysis itself and if reproducible, lends creedence to the result.

With air pollution as an example, the studies are used to inform major policy decisions and regulations. This can cost billions of dollars in implementation and are based on a small signal in the data using complex statistical methods. In this case, reproducibility is critical.

Reproducible Research: Concepts and Ideas (Part 2)

Research Pipeline

Start with measured Data and move to analytic data via processing code.

Move from analytic data to computational results via analytic code.

Move from computational results to figures, tables, and numerical summaries via presentation code.

Finally, move from figures, tables and summaries, to the article which is presented in a text form.

Essentially, we are saying that presentation code is not enough to reproduce the results, you need processing code, analytic code and presentation code in order to reproduce the results of the research.

So what do we need for our research to be reproducible?

- Analytic Data is available
- Analytic Code is available
- Documentation of code and data
- Standard means of distribution

Reproducible Research: Concepts and Ideas (Part 3)

Literate Statisical Programming

Sweave is one such program that uses Latex and R.

Knitr is an alternative and more recent R package. It includes many features that Sweave was lacking.

Scripting your Analysis

None

Structure of a Data Analysis (Part 1)

Steps:

- Define the question
- Define the ideal data set
- Determine what data you can access
- Obtain the data
- Clean the data
- EDA
- Statistical prediction/modeling
- Interpret results
- Chanllenge results
- Synthesize/write up results
- Create reproducible code

Start with a general question and make it concrete.

Can I automatically detect emails that are SPAM or not?

Can I use quantitative characteristics of the emails to classify them as Spam/Ham?

Define the ideal data set

The data set may depend on your goal

- Descriptive: a whole population

- Exploratory: a random sample with many variables measured
- Inferential: The right population, randomly samples
- Predictive: a training and test data set from the same question
- Causal: data from a randomized study
- Mechanistic: data about all components of the system.

Determine what data you can access

- Sometimes you can find data free on the web
- Other times, you may need to buy the data
- Be sure to respect the terms of use
- If the data don't exist, you may need to generate it yourself

Obtain the data

- Try to obtain the *raw* data
- Be sure to reference the source
- If you load the data from an internet source, record the url and time accessed.

Clean the data

- Raw data often needs to be processed
- If it is pre-processed, make sure you understand how that was done
- Understand the source of the data (if it was sampled, how?)
- May need reformating, subsampling, record these steps
- Determine if the data are good enough, if not, quit or change the data

Structure of a Data Analysis (Part 2)

Subsampling our data set.

Split the data set into a test and train set.

```
# Our data comes from the Kernlab package
library(kernlab)
```

```
## Warning: package 'kernlab' was built under R version 4.0.3
```

```r
data(spam)
# Perform the subsampling
set.seed(3435)
trainIndicator = rbinom(4601, size=1, prob=0.5)  # coin flip
table(trainIndicator)
```

```
## trainIndicator
##    0    1
## 2314 2287
```

```r
trainSpam = spam[trainIndicator == 1, ]
testSpam = spam[trainIndicator == 0, ]
```

Exploratory Data Analysis

- Look at summaries of the data
- Check for missing data
- Create exploratory plots
- Exploratory analysis

```r
head(trainSpam[,c(1:3)])
```

```
##    make address  all
## 1  0.00    0.64 0.64
## 7  0.00    0.00 0.00
## 9  0.15    0.00 0.46
## 12 0.00    0.00 0.25
## 14 0.00    0.00 0.00
## 16 0.00    0.42 0.42
```

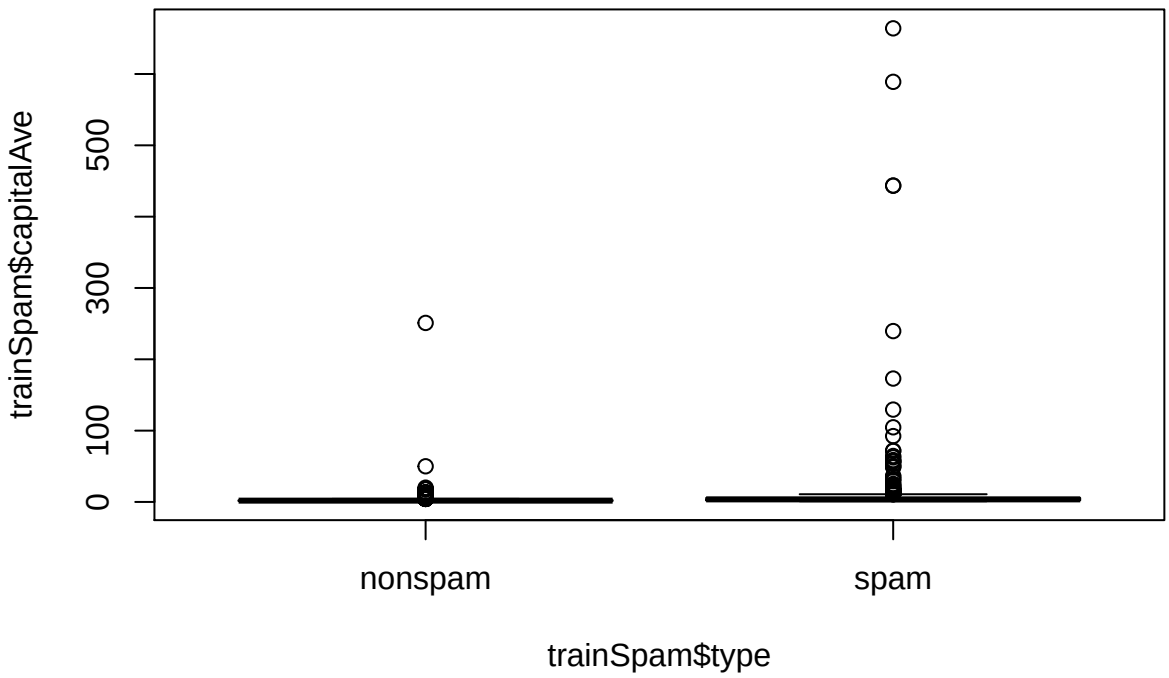
With this snapshot of the data, we can see that the columns are words that may or may not appear in the data and the rows are individual emails. The values here are the frequency with which the words appear in the email.

```r
table(trainSpam$type)
```

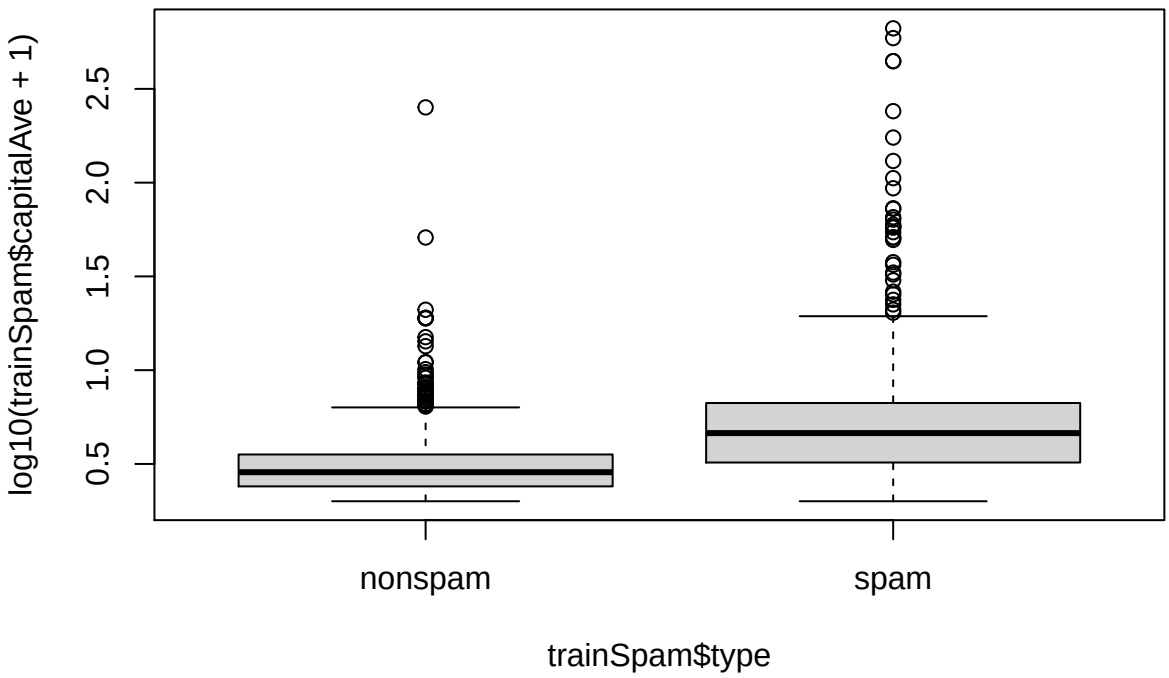
```
##
## nonspam    spam
##    1381     906
```

We can see here that there is a flag in the training data that tells us if the email was classified as spam or not.

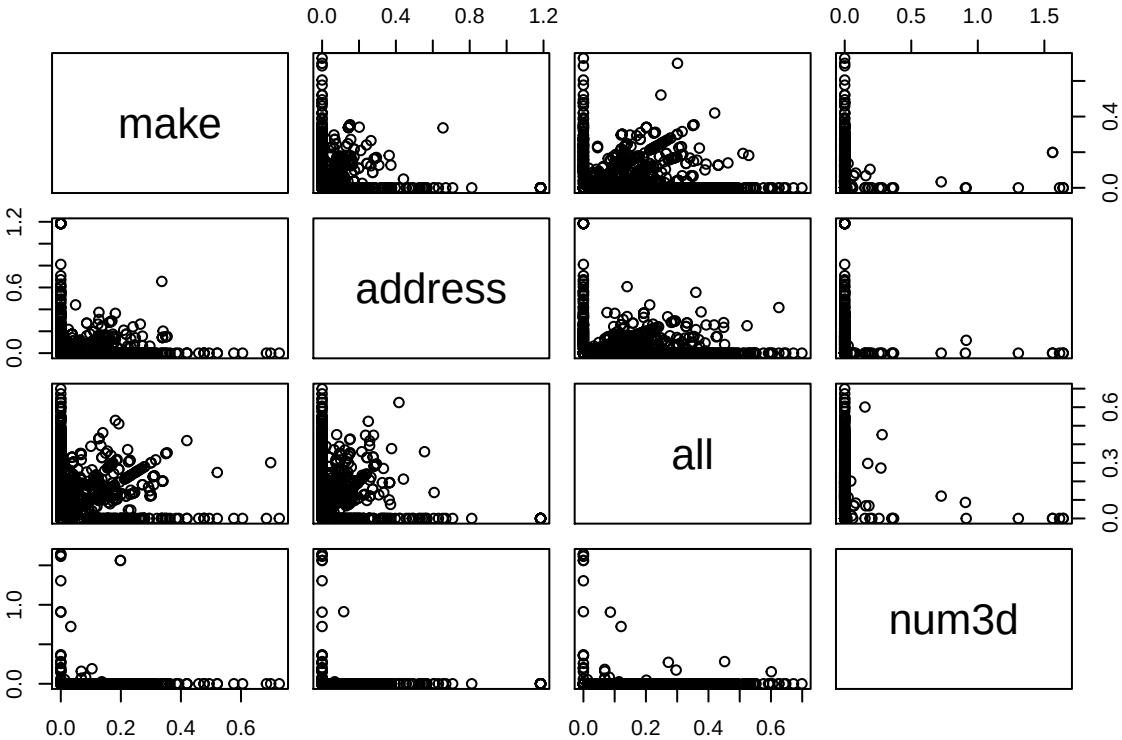
```r
plot(trainSpam$capitalAve ~ trainSpam$type)  # data is highly skewed, take log
```



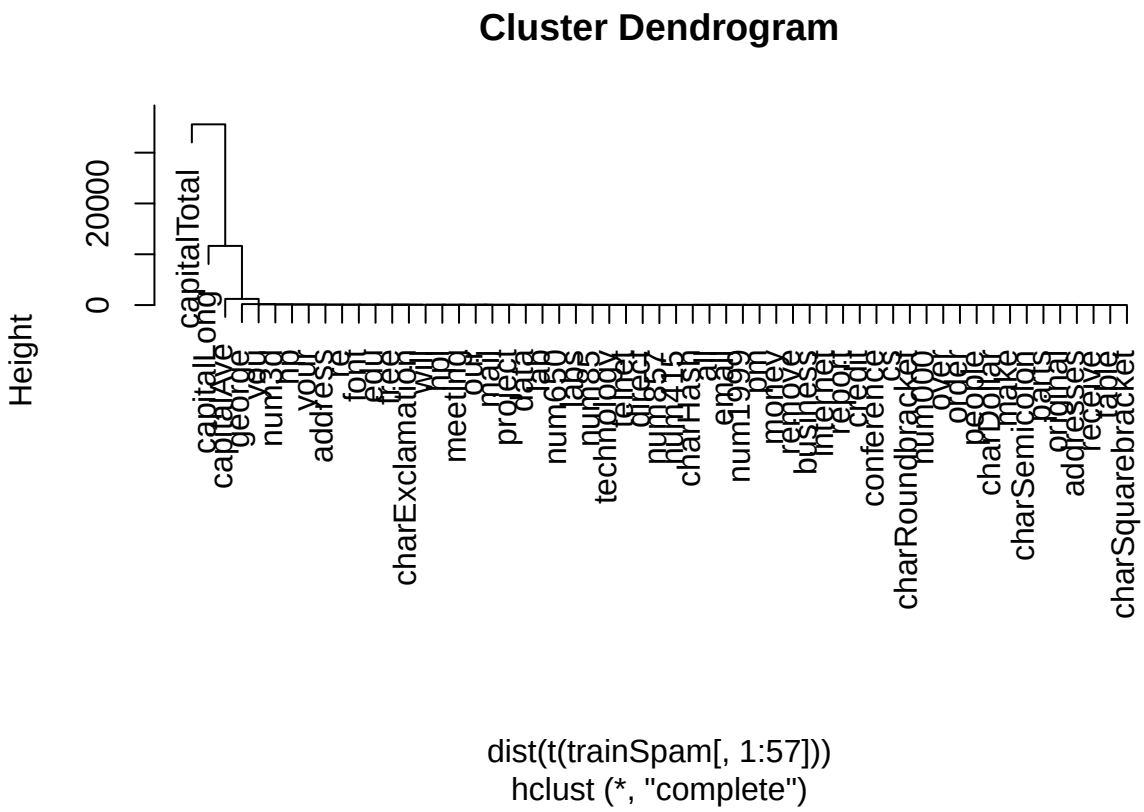

```r
plot(log10(trainSpam$capitalAve + 1) ~ trainSpam$type)  # Num of capital letters.
```




```r
plot(log10(trainSpam[, 1:4] + 1))  # Possible relationships between words
```

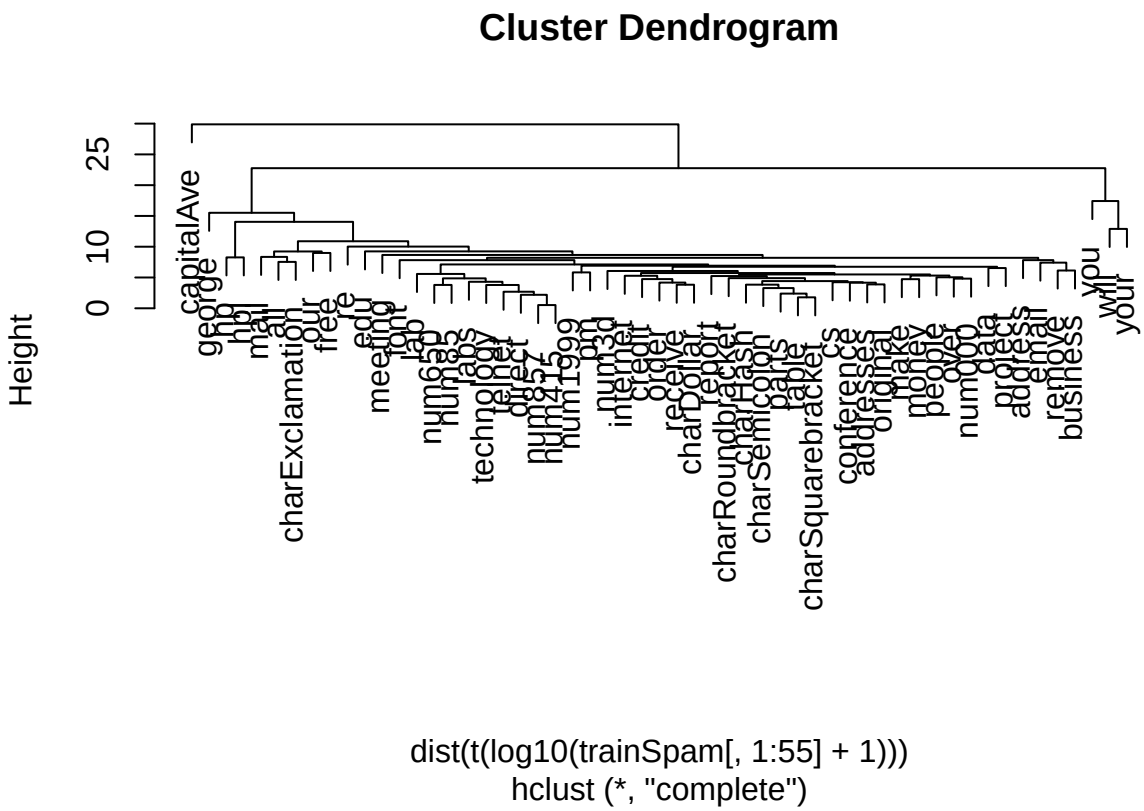



```
hCluster = hclust(dist(t(trainSpam[, 1:57])))
plot(hCluster)
```



Clustering like this is vulnerable to skewness in the data, but it can be redone with a transformation of the data.

```
# new cluster after transformation
# add one to avoid taking the log of 0
hClusterUpdated = hclust(dist(t(log10(trainSpam[, 1:55] + 1))))
plot(hClusterUpdated)
```



Statistical preditcion/modeling

- Should be informed by the results of your exploratory analysis
- Exact methods depend on the question of interest
- Transformations/processing should be accounted for when necessary
- Measures of uncertainty should be reported

Example: iterate through all of the words (variables) in the data set to see if we can use one of them as a single predictor of SPAM using a logistic regression.

```
trainSpam$numType = as.numeric(trainSpam$type) - 1
costFunction = function(x, y) {sum(x != (y > 0.5))}
cvError = rep(NA, 55)
library(boot)
for (i in 1:55) {
        lmFormula = reformulate(names(trainSpam)[i], response = 'numType')
        glmFit = glm(lmFormula, family = 'binomial', data = trainSpam)
        cvError[i] = cv.glm(trainSpam, glmFit, costFunction, 2)$delta[2]
}

# Which predictor has minimum cross-validated error?
names(trainSpam)[which.min(cvError)]
```

```
## [1] "charDollar"
```

Get a measure of uncertainty

```r
# USe the best model from the group
predictionModel <- glm(numType ~ charDollar, family = 'binomial', data = trainSpam)

# Get predictions on the test set
predictionTest <- predict(predictionModel, testSpam)
predictedSpam <- rep('nonSpam', dim(testSpam)[1])

# Classify as 'spam' for those with prob > 0.5
predictedSpam[predictionModel$fitted > 0.5] = 'spam'
```

In a logistic regression we don't get a binary 0 or 1 if a message was predicted to be spam, we get a probability that the message is spam, so we are assigning 'spam' to any message that got a probability over 0.5, or 50%.

```r
# classification table
table(predictedSpam, testSpam$type)
```

```
##
## predictedSpam nonspam spam
##       nonSpam    1346  458
##       spam         61  449
```

Error Rate = (61 +458) / (1346 + 458 +61 +449) = .2243, 22.4%

61 and 458 are errors. 61 was classified as spam but were not spam. 458 were classified as nonspam but were spam. Divide the sum of those two number by the sum of all four and you get the error rate.

Interpret Results

- Use the appropriate language
  - Describes
  - Correlates with/ associated with
  - Leads to/causes
  - Predicts

- Give an explanation

- Interpret coefficients

- Interpret measures of uncertainty

It's important to only use language that actually describes what has been done and what we can assume. Don't use "causes" if you can't imply causation.

Our example:

- The fraction of characters that are dollar signs can be used to predict if an email is spam.

- Anything more than 6.6% dollar signs is classified as spam

- More dollar signs always means more spam under our prediction

- Our test set error rate was 22.4%

Challenge Results

- Challenge all steps:
  - Question
  - Data Source
  - Processing
  - Analysis
  - Conclusions

- Challenge measures of uncertainty

- Challenge choices of terms to include in models

- Think of potential alternative analysis

Synthesize/write up results

- Lead with the question

- Summarize the analyses into the story

- Don't include every analysis, include it:
  - if it is needed for the story
  - If it is needed to address a challenge

- Order analyses according to the story, rather than chronologically

- Include "pretty" figures that contribute to the story

In our example:

- Can we use quantitative characteristics of the emails to classify them as Spam/Ham?

- Collected data from UCI -> created training/test sets, explored relationships, choose logistic model on training set by cross validation, applied to test, 78% test set accuracy

- Number of dollar signs seems reasonable

- 78% isn't that great, I could use more variables, why logistic regression?

Create reproducible code

Organizing Your Analysis

Data Analysis Files:

- Data
  - Raw data
  - Processed data

- Figures
  - exploratory figures
  - final figures

- R code
  - raw / unused scripts
  - final scripts
  - r markdown files

- Text
  - README files
  - text of analysis / report

Raw data

Should be stored in your analysis folder. If accessed from the web, include the URL, description, and date accessed in README.

Processed Data

Should be named so it is easy to see which script generated the data. The processing script - processed data mappin should occur in the README. Should be tidy.

Exploratory figures

Figures made during the course of the analysis, not necessarily in the final report. They do not need to be "pretty."

Final Figures

Usually a small subset of the original figures. Axes/colors set to make the figure clear. Possibly multiple panels.

Raw / Unused Scripts

May be less commented (but comments help), multiple versions, may include analyses that are later discarded.

final Scripts

Clearly commented. Small comments liberally, bigger commented blocks for whole sections. Include processing details. Only analyses that appear in the final write up.

R Markdown Files

Can be used to generate reproducible reports. Text and R code can be integrated. Very easy to create in RStudio.

README

Not necessary if you use R Markdown. Should contain step by step instructions for analysis.

Text of analysis / Report

Should include title, introduction (motivation), methods (stats you used), results (including measures of uncertainty), and conclusions (including potential problems). Should tell a story. Should not include every analysis you performed. References should be included for statistical methods.

Project template R package can be used for a cookie cutter approach to projects.